

CS247 – SPRING 26 – TUTORIAL 3

M5 – MODULES

M6 – INTERFACE SPECIFICATIONS

The Tutorial's Quick Tutorial

1. The topics covered by Prof. Victoria on her slides will be further expanded during our meetings.
2. Each topic will have up to 3 subsections:
 1. A quick recap;
 2. an **Example**;
 3. a **Challenge**;
3. We will use Microsoft Copilot Chat (<https://office.com/chat>), specially with the proposed **Challenges**, to evaluate how AI behaves with these “*gotcha!*” questions, so we can learn to evaluate AI’s strengths and shortcomings.
4. Please, review UW’s “**Responsible Use of AI Tools with University Data**” guidelines at <https://uwaterloo.ca/information-systems-technology/about/policies-standards-and-guidelines/artificial-intelligence-ai/responsible-use-ai-tools-university-data>
5. Please, review the course’s guidelines regarding the usage of AI at https://outline.uwaterloo.ca/viewer/view/nye4ps#generative_ai

Supporting Material

Repo: <https://github.com/talesmp/CS247-S26-Tutorial3.git>

Vevox Session

Join at:
vevox.app

ID:
448-918-461

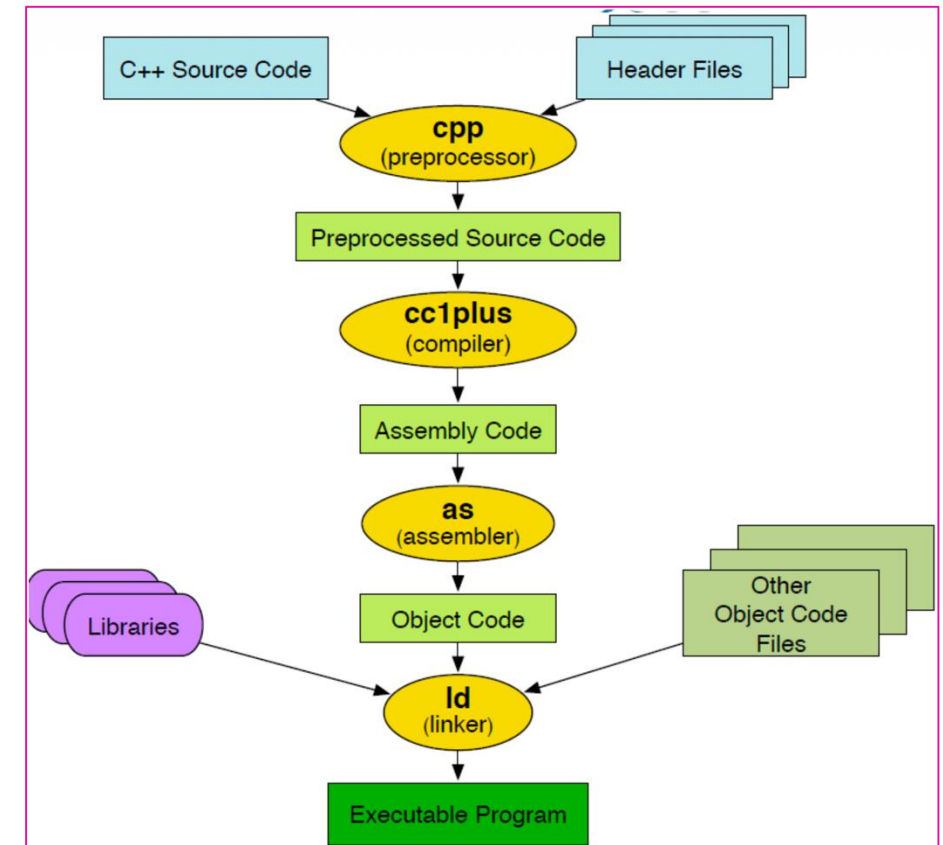


The Thread

DECOMPOSE → CONNECT → BUILD → AUTOMATE → SPECIFY → DEBUG

“Which tool produced this error?”

- **Preprocessor** → pastes text, blindly
- **Compiler** → checks **one file** at a time
- **Linker** → counts definitions
 - **0 providers** = *undefined reference*
 - **2 providers** = *multiple definition*



The Thread

DECOMPOSE → CONNECT → BUILD → AUTOMATE → SPECIFY → DEBUG

DECOMPOSE	CONNECT	BUILD	AUTOMATE	SPECIFY	DEBUG
1	2~5	6~7	8	9~11	12~13
why split?	headers / macros / cycles	the 4 stages + errors	make and Makefile	header IS a contract	when the contract breaks

Phase 1 – DECOMPOSE & CONNECT

Making the split files aware of each other.

DECOMPOSE → CONNECT → BUILD → AUTOMATE → SPECIFY → DEBUG

1. Program Decomposition (Separate Compilation)

A program is **an idea written as code**.

This idea can potentially be written as a single file (a monolith).

But breaking a monolith into modules has four advantages:

1. **parallel development;**
2. **separate compilation;**
3. **reuse;**
4. **incremental development.**

So, a program is **one idea split across many** files. The goal today is to make that split **safe**.

DECOMPOSE → CONNECT → BUILD → AUTOMATE → SPECIFY → DEBUG

1. Program Decomposition (Separate Compilation)

The catch: under *separate compilation* the compiler sees **one translation unit (TU) at a time, in isolation.**

Rule: anything you use must be declared or defined before use.

So when `main.cpp` writes `B *b_obj;` but class `B` lives in `ADT1.cpp`, the compiler has *never heard of B*.

2. Global Declarations (Declaration vs Definition)

A **declaration** announces a name exists and gives its *shape* $\Rightarrow$ may repeat many times if identical.

A **definition** brings the thing into being: allocates storage/emits code $\Rightarrow$ **exactly once** (One Definition Rule, or **ODR**).

Every definition is a declaration; not vice versa.

2. Global Declarations (Declaration vs Definition)

- `extern int i;` $\Rightarrow$ pure *variable declaration* (the variable-equivalent of a function prototype); one `.cpp` holds `int i = 0;` $\Rightarrow$ the definition
- `const` defaults to **internal linkage** (compiler substitutes the value, allocates no storage, exports no symbol) $\Rightarrow$ not shareable across files unless you opt in with `extern const`.
- **Linkage** = how far a name reaches:
 - **internal**: private to its TU, default for `const`
 - **external**: one entity shared program-wide $\Rightarrow$ default for globals/functions

3. Header Files & The Preprocessor

Put the declarations other modules need into **one header** and `#include` it everywhere (single source of truth), instead of hand-copying declarations into every `.cpp`.

`#include` is **blind text copy-paste**, performed by the **preprocessor**.

A separate pass *before* the compiler that only touches `#` lines. The four directives:

- `#include` => paste a file,
- `#define` => a bare flag, or a valued macro => find-and-replace,
- `#ifdef/#ifndef ... #endif` => conditional compilation: keep/delete a block.

3. Header Guards & Duplicate Inclusions

Because `#include` is blind pasting, the *same* header can land in one TU twice (e.g. `main.cpp` includes both `ADT1.h` and `ADT2.h`, and `ADT2.h` itself includes `ADT1.h`).

Identical *declarations* may repeat, but a **class definition cannot appear twice in one TU** => "redefinition" error.

Fix: wrap every header in a **header guard**:

```
#ifndef RATIONAL_H
#define RATIONAL_H
// ... header body ...
#endif
```

4. Macros (Preprocessor Directives)

Macros paste *text*, not *values*.

There's no expression evaluation or parenthesization.

This is exactly why modern C++ prefers real `const`/`constexpr`/`inline` functions over macros.

The preprocessor is the mechanism that matters here, but it's a sharp tool.

5. Circular Dependencies and Forward Declarations

```
$ g++ main.cpp -o app
```

```
error: 'Course' does not name a type ← COMPILER
```

- `course.h` $\Leftrightarrow$ `student.h` include each other
- Guards stop the **loop**, not the **ordering**

Phase 2 – BUILD & AUTOMATE

Turning the split files into one program.

DECOMPOSE → CONNECT → BUILD → AUTOMATE → SPECIFY → DEBUG

6. The Build Pipeline

`g++ -E hello.cpp # 1 preprocessor` $\Rightarrow$ pasted text

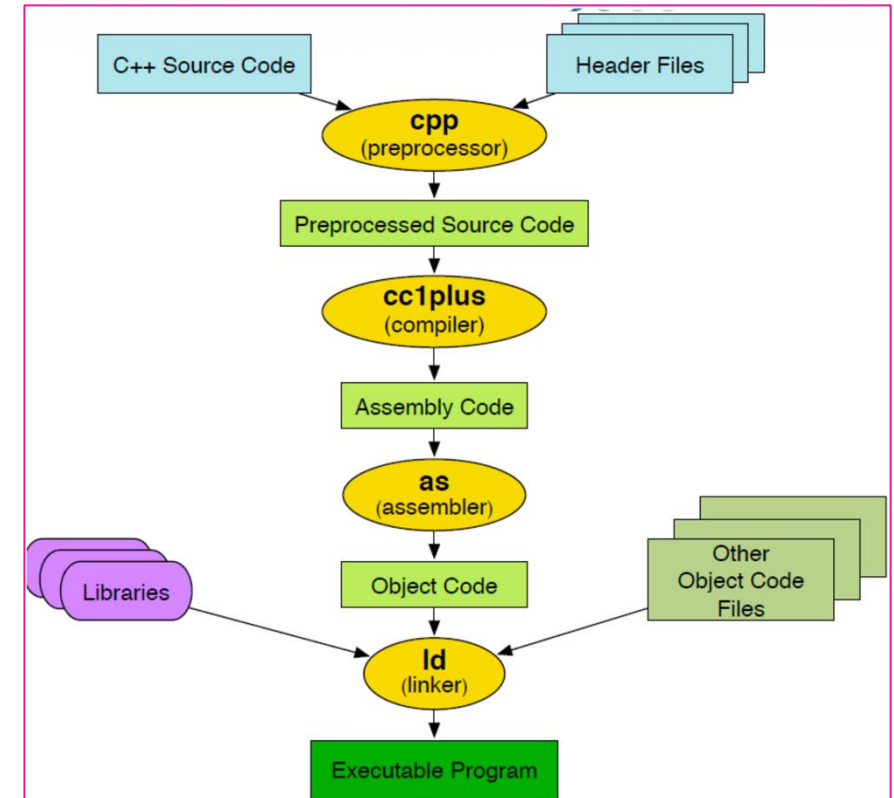
`g++ -S hello.cpp # 2 compiler` $\Rightarrow$ assembly

`g++ -c hello.cpp # 3 assembler` $\Rightarrow$ hello.o

`nm hello.o` # T = provides ; U = needs

`g++ hello.o` # 4 linker $\Rightarrow$ executable

- Four tools: **cpp** ; **cc1plus** ; **as** ; **ld**



7. Undefined Reference

```
$ g++ main.cpp -o app      # forgot mathutil.cpp!  
undefined reference to 'square(int)' <= LINKER
```

- The code is **correct** $\Rightarrow$ a definition never reached the link
 - *multiple definition* = 2 providers
 - *undefined reference* = 0

8. Make & Makefile

How a Makefile works => Variables & Flags

```
CXX = g++                                # which compiler
CXXFLAGS = -std=c++17 -Wall -g -MMD      # flags passed to it
EXEC = myprogram                        # output name (your name of choice)
CCFILES = $(wildcard *.cc)              # => every .cc in the folder
OBJECTS = ${CCFILES:.cc=.o}             # => same list, .cc swapped for .o
DEPENDS = ${CCFILES:.cc=.d}             # => same list, .cc swapped for .d
```

- Variables (make calls them macros):
 - Define a setting once, reuse via `${VAR}` everywhere.
 - Change compiler or flags in one place.
 - `${VAR}` and `$(VAR)` are identical.
- CXX/CXXFLAGS are special names make's built-in rules already use.
 - `-std=c++17` # pick the C++ standard (14 / 17 / 20...)
 - `-Wall` # turn on all common warnings => catches bugs early
 - `-g` # embed debug info → gdb/valgrind can show source lines
 - `-MMD` # while compiling, also write a .d file listing that file's header dependencies
- `$(wildcard *.cc)` lists files;
- `${CCFILES:.cc=.o}` is a substitution (.cc => .o).

8. Make & Makefile

How a Makefile works => Rules & Rebuild Sequence

```
${EXEC}: ${OBJECTS}                # target : prerequisites
    ${CXX} ${OBJECTS} -o ${EXEC}    # recipe => line MUST start with a TAB
```

```
-include ${DEPENDS}                # pull in the auto-generated .d rules
```

```
.PHONY: clean                      # 'clean' is a label, not a file
clean:
    rm -f ${EXEC} ${OBJECTS} ${DEPENDS}
```

The sequence: how make decides (timestamps):

1. `make` => build `${EXEC}` => it needs every `${OBJECTS}`.
2. For each `.o`: **missing, or older than its `.cc/.h`**? => Recompile it (implicit rule).
3. Those `.h` dependencies come from the `.d` files => courtesy of `-MMD`.
4. Any `.o` newer than `${EXEC}`? => **relink**.
5. Already up to date => **do nothing**. Only what changed is rebuilt.

8. Make & Makefile

```
CXX = g++                                # which compiler to use
CXXFLAGS = -std=c++17 -Wall -g -MMD      # -MMD auto-generates .d dependency files
EXEC = vecdemo
CCFILES = $(wildcard *.cc)               # every .cc file in this directory
OBJECTS = ${CCFILES:.cc=.o}              # same names, .cc -> .o
DEPENDS = ${CCFILES:.cc=.d}              # same names, .cc -> .d

${EXEC}: ${OBJECTS}                      # the executable depends on all the .o files
    ${CXX} ${OBJECTS} -o ${EXEC}         # recipe (TAB-indented): link them

-include ${DEPENDS}                      # pull in the auto-generated dependency rules

.PHONY: clean
clean:
    rm -f ${EXEC} ${OBJECTS} ${DEPENDS}
```

Three things that silently break it:

- recipe indented with spaces instead of a **TAB** => missing separator;
- a mistyped variable like **CCFILESs** ≠ **CCFILES** => expands to empty;
- **rm** without **-f** => errors when files are already gone.

8. Make & Makefile

Incremental rebuilds

```
$ make                # builds
```

```
$ make                # "up to date" — does NOTHING (timestamps)
```

```
$ touch vec.cc && make # rebuilds ONLY vec.o, then relinks (main.o reused)
```

```
$ touch vec.h && make  # rebuilds BOTH main.o AND vec.o (both depend on vec.h), then relinks
```

```
$ make -f Makefile.broken
```

```
*** missing separator. Stop.      <= recipe needs a TAB
```

Phase 3 – SPECIFY

A header isn't just text. It's a contract.

DECOMPOSE → CONNECT → BUILD → AUTOMATE → SPECIFY → DEBUG

9. Interfaces as contracts

A header is an **interface** $\Rightarrow$ a **contract**:

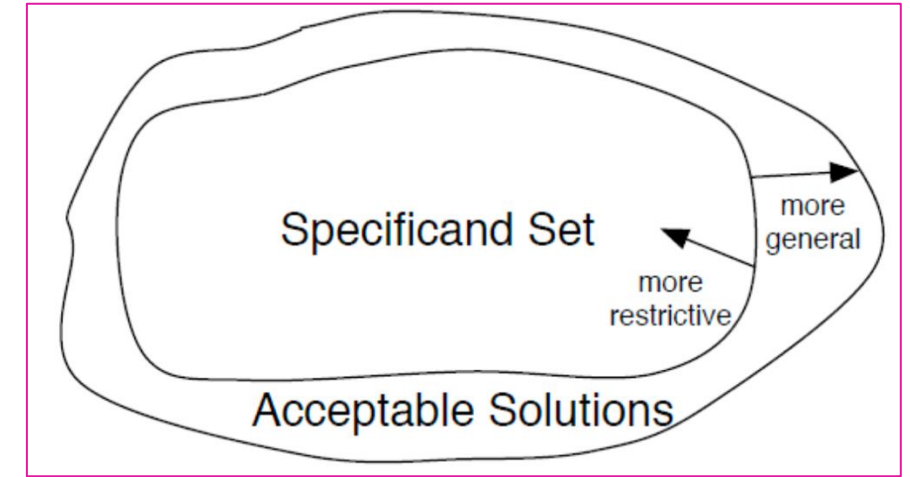
- **requires** $\Rightarrow$ client's duty
- **provides** $\Rightarrow$ provider's guarantee
- Violate a precondition $\Rightarrow$ **anything goes** (and that's a *feature*)
- A bad case is **requires** **or** **throws** $\Rightarrow$ **never both**

10. Specifying Derived Classes

- **Specification fields** model observable state (not private members)
- Override written as "*base + a bit extra*" drops clauses
- **An override must restate the COMPLETE spec**

11. Specificand Sets

Spec	A	B	C
1: requires present	✓	✓	✓
2: else -1		✓	
4: else out-of-range		✓	✓



present 20 $\Rightarrow$ A=1 B=1 (smallest) C=3 (largest)

absent 99 $\Rightarrow$ A = UB (garbage/crash) B=-1 C=5

Phase 4 – DEBUG

DECOMPOSE → CONNECT → BUILD → AUTOMATE → SPECIFY → **DEBUG**

- Linux servers: `gdb` / `valgrind`
- Mac (my case): `lldb`

12. gdb

- Interactive debugger: run your program under its control, pause it, inspect it.
- Compile with `-g` first => embeds the source-line info gdb needs.
 - `run` => reproduce the crash .
 - `bt` (backtrace) => the call stack that led here
 - `p` (print x) → a variable's value.
- Also: set a breakpoint, next/step through line by line, continue.
- Finds where it died and helps you reason why.
- **Remember: the *crash site* ≠ the *bug*.**

13. valgrind

- Runs your program on a synthetic CPU, watching every memory access and allocation.
- `valgrind ./prog` (Memcheck by default)
 - add `--leak-check=full` for leak detail
 - compile with `-g` so it cites exact source lines.
- Reports *Invalid read/write, definitely lost, uninitialised value* => each tied to a line number.
- **The lesson: “runs fine” ≠ “correct”** => It surfaces the bugs your program won't tell you about.

Conclusion

A program is **one idea split across many files**.

The goal today is to make that split **safe**.

header = interface = contract

- declarations / headers / preprocessor => **connect** the split files
- compiler + linker + **make** => turn the split files into a program
- a stable contract lets the hidden implementation **change freely**
- **gdb** / **valgrind** show what a broken contract did at runtime

Every error named its tool: **preprocessor pastes** => **compiler checks one file** => **linker counts definitions**

UNIVERSITY OF
WATERLOO



FACULTY OF MATHEMATICS
QUESTIONS?

THANK YOU!

tmellopaiva@uwaterloo.ca